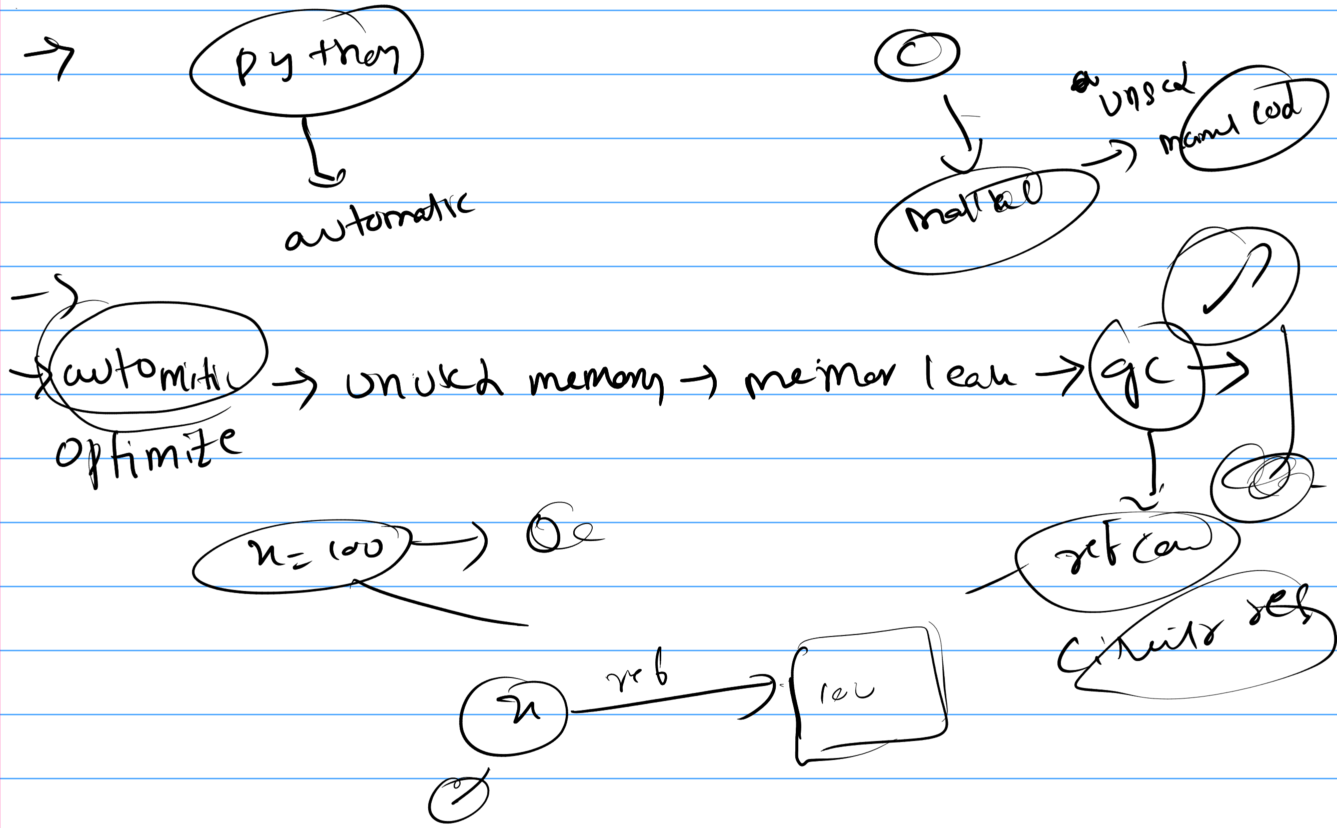
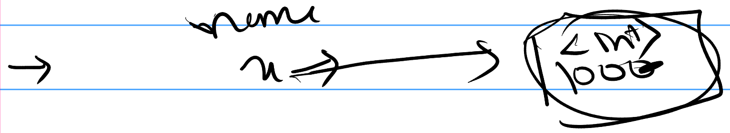
python-Day-20

what is memory management ?


python
automatic
C
malloc
unused memory code
automatic
optimize
unused memory
memory leak
gc
ref count
circular ref
x = 100
0
x
ref
100



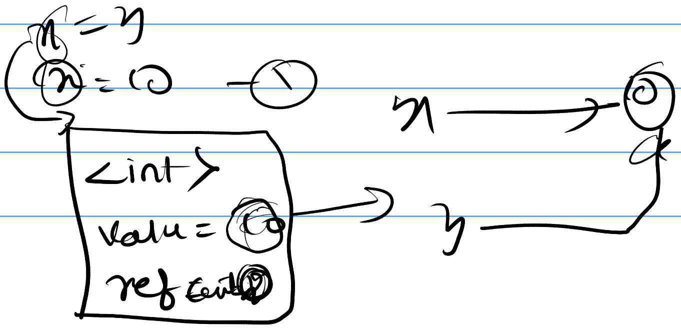
→ by using the id() → it give obj location

== vs is

== → it check the value of object

is → obj id (memory id check) → check

object

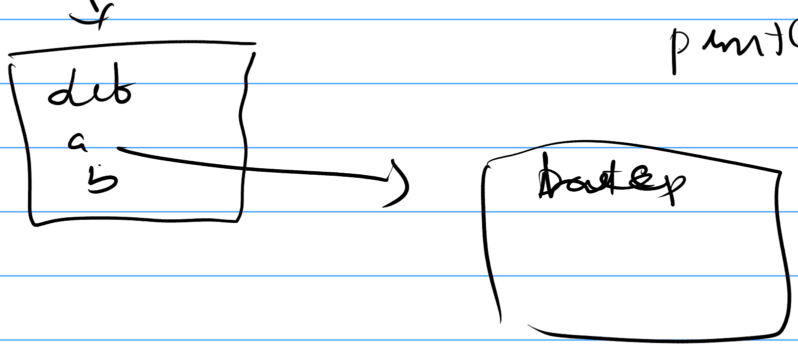

x = y
x = 10
<int>
value = 10
ref count 2
x
y

Stack

→ function call local

mainfarein() → ~~main~~ thare (fun)

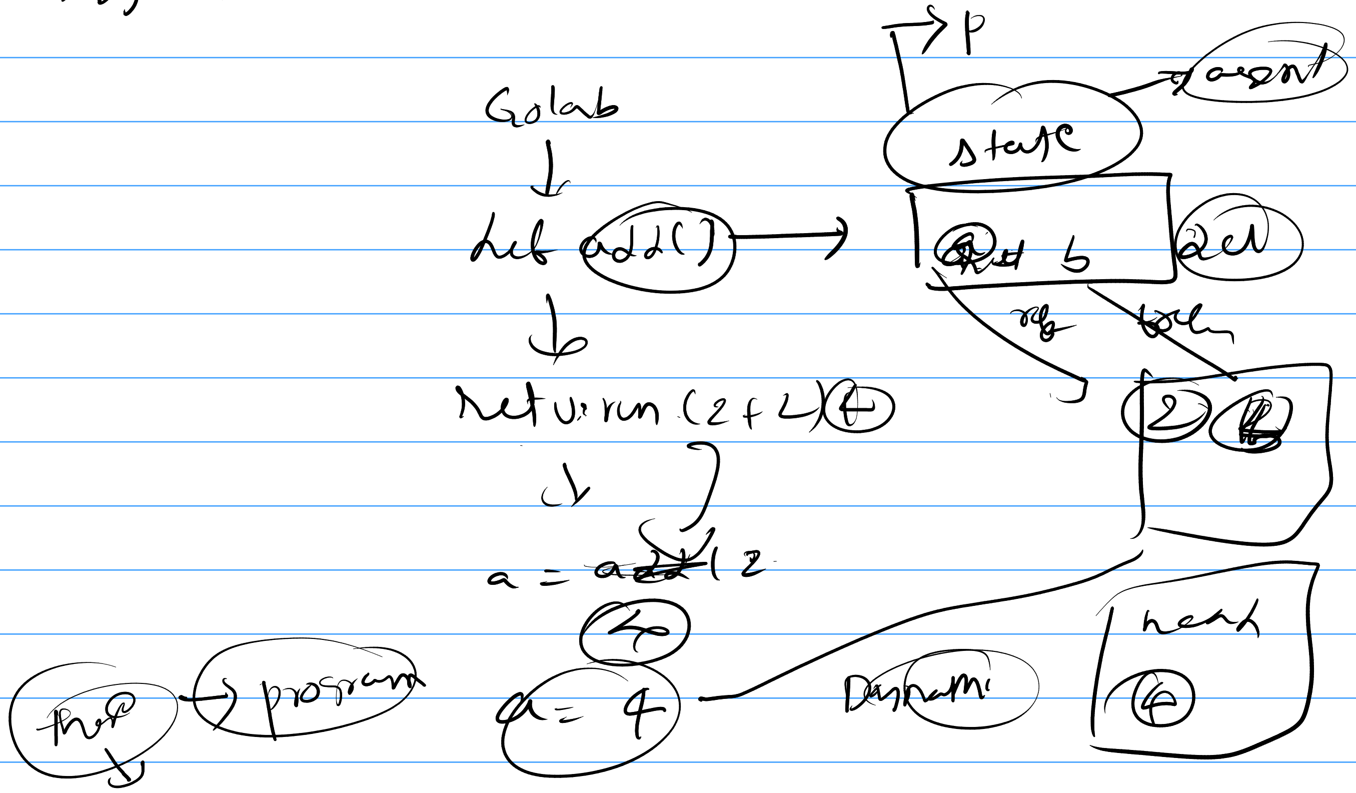
def add(a,b):
    result = a+b
    return result
~~K~~ = add(2,2)
print(a)
④



def
a
b
~~Dat~~ Heap

→ heap

→ Dynamic Data allocation



Golab
def add()
b
Return (2+2) ④
a = ~~add~~ (2
④
a = 4
thre
program
p
state
accont
add b
del
ref
token
2
heap
④
Dynamic

Stack → global

heaps

function allow
→ state rememb

Stored the object in
run time

Reference Counts:

→ obj → How many variable referred

a = 10
b = a
c = a
print(c) # 10

a = [1, 2, 3]
b = a
c = a ✗
c.append(4) ✓
print(a) →
[1, 2, 3, 4]

a = [1, 2, 3]
b = [1, 2, 3]
b.append(a) → [1, 2, 3] ✓
print(b) # [1, 2, 3, 1, 2, 3]
[1, 2, 3, [1, 2, 3]]

iterable

Dictionary

→ sys → getrefcount → ① ③ +1
(4) → ③
④ capem

→ append → iter append

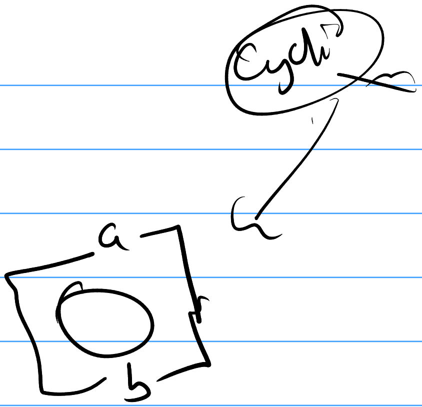
→ ref → obj → gc → active → 0 →

→ gc → automatic

cercular ref :

a = [ ]
b = [ ]
a.apen(b)
b.apen(a)

c
b

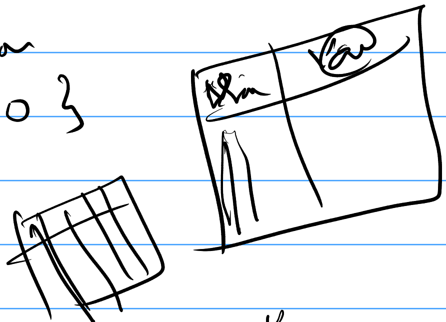


→ cyclic garbar collecte
↓

G(→)
↓
Garbage colecter
↓
automaticalli delet unused obj or refr

D →
→ gc.get_collect( )

a-
aD{key: valu
↓
~~Hash~~ Hash va
10 —
key va
a = { 10 }



10
a — name = "guru"
print(a[nam])

immutable and mutable

a = [1,2,3]
b = a
print(a is b)
True

a = [1,2,3]
b = [1,2,3]
print(a is b)
False

a = 10
b = 10
print(a is b)
~~True~~
False
(R →)

a = 10 \ []
b = a
print(a is b)
→ P
False
True → J
T → g